Supplementary Material for
CDS4RAG: Cyclic Dual-Sequential Hyperparameter Optimization for RAG

Pengzhou Chen[1*], **Tao Chen**[2†]
[1]School of Computer Science and Engineering, UESTC, Chengdu, China
[2]IDEAS Lab, University of Birmingham, Birmingham, UK
t.chen@bham.ac.uk

A More Correlations Between Retrieval and Generation Quality

Figure 1 presents the Pearson correlations between the quality of retrieval and generation over different RAG hyperparameter configurations using given queries on all cases studied. This extends the Figure 2 from the main paper. We see that there exist a good extent of positively monotonic correlation in general (with mostly moderate to strong signal), which enables `CDS4RAG` to yield considerable advances over the state-of-the-art algorithms, thanks to the concept of cyclic dual-sequential hyperparameter optimization. The only exception is that RAG with `Qwen3-8B` exhibits a relatively unclear correlation on the `BioASQ` benchmark, which is consistent with our experimental results, as in that case, the superiority of `CDS4RAG` is more blurred than in the other LLMs/benchmarks.

B Additional Experiment Setup Details

B.1 RAG Prompt Template

To ensure a fair comparison, we employ a unified prompt template across all algorithms. As can be seen from Figure 2, the prompt consists of a fixed system instruction and a user input that incorporates the retrieved documents and the query. The content highlighted by $\{\cdots\}$ would be replaced by the actual query details and contexts retrieved.

B.2 Evaluation Metric Details

We evaluate the generation quality using the token-level F1-score, which measures the overlap between the RAG generations and the ground truth answers. For a given pair consisting of the generation y from a query and a reference r, let $T(y)$ and $T(r)$ denote the multisets of tokens derived from them. Specifically, for RAG with `Qwen-3-8B`, we explicitly exclude the generated reasoning process (chain-of-thought) from y and only calculate the score based on the final answer. The number of matched tokens is calculated based on the multiset intersection:

$$N_{\text{match}} = \sum_{w \in V} \min(\text{count}(w, T(y)), \text{count}(w, T(r))) \quad (1)$$

*Pengzhou Chen is also supervised in the IDEAS Lab.
†Tao Chen is the corresponding author.

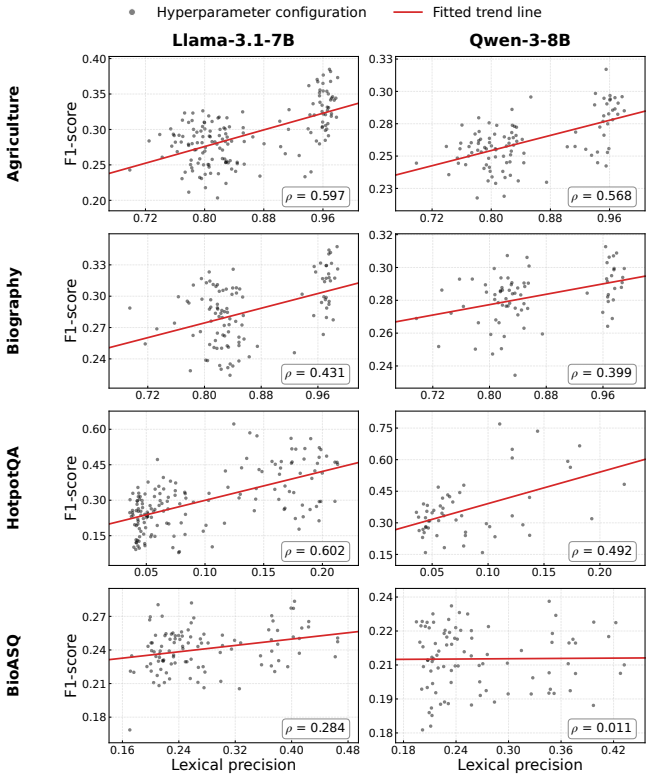

Figure 1: Full correlation analysis between retriever and generator quality for RAG over all cases studied (an extension of the Figure 2 in the main paper). The Pearson correlation is denoted by ρ.

where V is the vocabulary of unique tokens, and $\text{count}(w, S)$ is the frequency of token w in multiset S. The Precision (P) and Recall (R) for this pair are defined as:

$$P = \frac{N_{\text{match}}}{|T(y)|}, \quad R = \frac{N_{\text{match}}}{|T(r)|} \quad (2)$$

The F1-score for the individual query is the harmonic mean of precision and recall:

$$\text{F1} = 2 \cdot \frac{P \cdot R}{P + R} \quad (3)$$

To evaluate the overall performance on the given batch of

Query Template

[System Message]
You are a helpful, respectful and honest assistant. Always answer as helpfully as possible, while being safe. Your answers should not include any harmful, unethical, racist, sexist, toxic, dangerous, or illegal content. Please ensure that your responses are socially unbiased and positive in nature. If you need include the answer just output the answer, no need to explain.

[User Message]
`[document]:` {`retrieved context`}
`[conversation]:` {`question`}. Answer solely based on the provided context. If information is absent, state 'I do not know'.

Figure 2: The unified prompt template for RAG.

Method	Hyperparameter Values	Description
HEBO	Model = GP; Acq = MACE; Opt = NSGA-II; Initial size = 5	Heteroscedastic evolutionary BO using MACE acquisition function.
BO	Model = GP; Acq = EI; Initial = Max-min; Initial size = 5	Standard Bayesian Optimization with Gaussian Process surrogate.
TPE	Initial size = 5; $\gamma = 0.25$;	Use kernel density estimation to distinguish good and bad samples.
RayTune	Model = TPE; Initial size = 5; $\gamma = 0.25$;	Tailor Tree-structured Parzen Estimator with default settings.
AutoRAG-HP	Space = Discrete; Quantization levels = 5	Use Hierarchical Multi-Armed Bandit on discretized parameters.
Greedy	Space = Discrete; Quantization levels = 5	Optimize iteratively over a finite candidate set.
Random	N/A	Performs uniform sampling with distinct seeds.
CDS4RAG	$\alpha = 0.5$; $\beta = 2$; $N = 10$	Optimize via a new formulation of cyclic dual-sequential problem.

Table 1: Detailed specifications and configurations of the baseline algorithms used in the experiments.

queries $\mathcal{Q}$ consisting of n queries, we calculate the average F1-score across all given queries:

$$\text{F1-score} = \frac{1}{n}\sum_{i=1}^{n} \text{F1}_i \tag{4}$$

where F1_i denotes the F1-score calculated for the i-th query-generation pair in the dataset.

B.3 Parameter Settings for Algorithms

We delineate the detailed configurations of the counterpart algorithms and CDS4RAG in Table 1. For Random Search, hyperparameter configurations are sampled uniformly from the continuous search space using distinct random seeds to ensure reliability. Regarding Greedy Search and AutoRAG-HP, we adapt the experimental setting by discretizing all continuous hyperparameters. This is necessary because AutoRAG-HP originally operates on discrete variables, while Greedy Search requires a finite candidate set for iterative optimization. Consequently, a unified discrete search space is employed for these methods.

For other baselines, RayTune employs the Tree-structured Parzen Estimator (TPE) with a splitting threshold of $\gamma = 0.25$. Bayesian Optimization (BO) utilizes a Gaussian Process (GP) surrogate with Expected Improvement (EI) as the acquisition function, initialized by maximizing the internal minimum distance. HEBO leverages a GP surrogate with Multi-Armed Contextual Bandit Estimation (MACE) and NSGA-II as the optimizer. To maintain a fair comparison with CDS4RAG, the number of initial points is set to 5 for all methods requiring an initialization phase.

C Additional Results

Here, we provide detailed trajectories of the results per case that were omitted from the main paper due to space constraints.

C.1 Optimization Trajectories for Comparing CDS4RAG against vanilla Algorithms

As in Figure 3, we show the optimization trajectories of the F1-score over a 60 minutes period and compare the proposed CDS4RAG indicated by dashed lines integrated with three SOTA optimization algorithms—HEBO, BO, and TPE—against the standard vanilla versions of these algorithms indicated by solid lines. Clearly, the CDS4RAG variants generally converge faster and achieve higher F1-score compared to their corresponding counterparts with better speedup under limited budget, hence saving the cost, particularly in the HotpotQA benchmark.

C.2 Optimization Trajectories for Ablation Study

The trajectories for the ablation study has been shown in Figure 4, where we replace the within-cycle budget provisioning with fixed, equally dividing budgets to retriever and generator, denoted as w/o pro; we also remove the cross-cycle generator seeding, denoted as w/o seeding. Again, the experiments are with two LLMs across all four benchmarks with 10 runs per setting as before. Overall, the ablation results demonstrate that our CDS4RAG consistently enhances the performance of both models across most tasks with improved speedup and time/cost saving, even considering some limited budget scenarios.

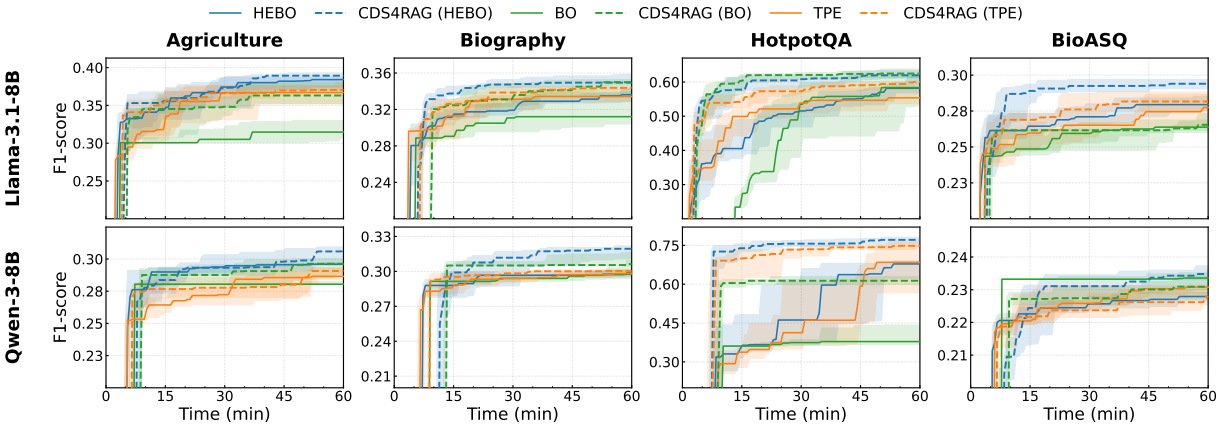

Figure 3: Comparing CDS4RAG paired with HEBO, BO, and TPE against their vanilla original versions for optimizing RAG over 10 runs.

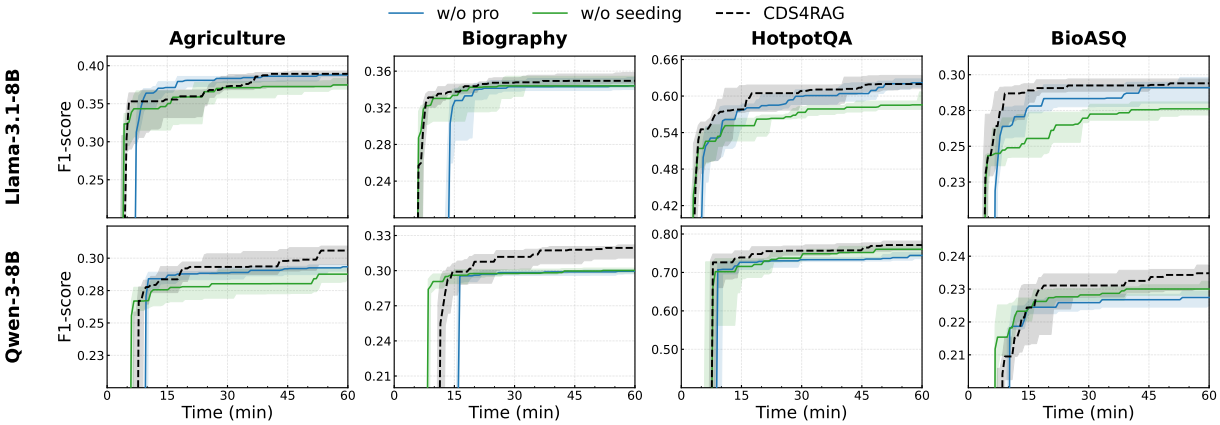

Figure 4: Ablation results on CDS4RAG over 10 runs.

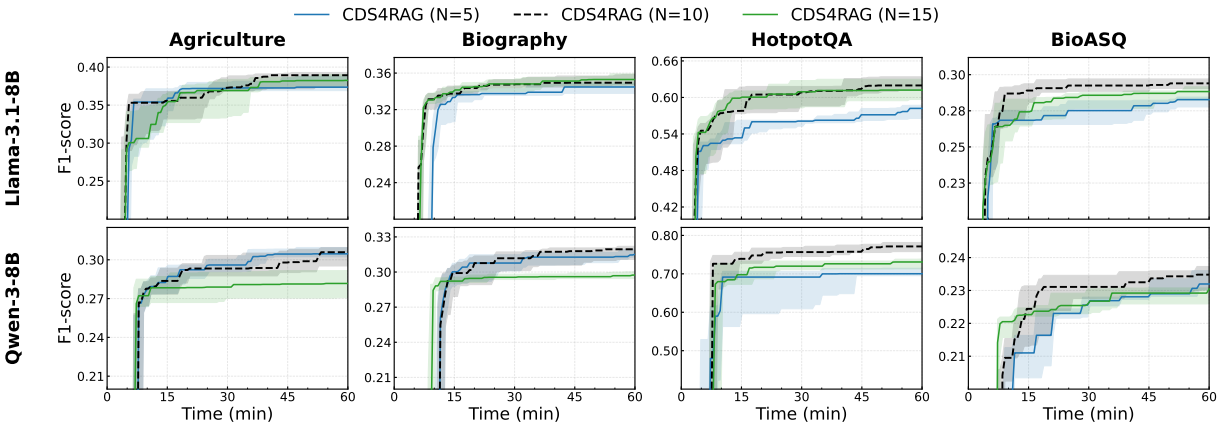

Figure 5: Sensitivity of CDS4RAG to N over 10 runs.

C.3 Sensitivity of CDS4RAG to N

We conduct a sensitivity analysis on the fixed number of evaluations N for the generator. To that end, we set $N \in \{5, 10, 15\}$ From Figure 5, clearly, the empirical results are well-aligned with our theory: a too large N would reduce the contribution from optimizing/exploring the retriever, while a too small N can cause insufficient exploitation in optimizing the generator. Among others, $N = 10$, which is our default, serves as a sweet point that leads to the best results, which we have set as default in this work.

C.4 Holdout Test Queries Validation

We evaluate the effectiveness of all methods on the holdout test queries from *HotpotQA* and *BioASQ*, using two backbone LLMs, `Llama-3.1-8B` and `Qwen-3-8B`. The *Agriculture* and *Biography* benchmarks are excluded due to the small sample size of their holdout sets, which is insufficient for reliable evaluation. To maintain consistency with the number of queries used during hyperparameter optimization, we construct the test sets by selecting an additional set of queries of matching size. For each method, we fix the best configuration obtained during optimization and test if under the holdout queries; we report the mean and standard deviation of F1 across 10 repeated runs. As shown in Tables 2 and 3, the `CDS4RAG` variants consistently outperform their corresponding single-stage optimizers across both datasets and backbone models. Notably, the improvement is most pronounced on *HotpotQA*, while gains on *BioASQ* are smaller but consistent, demonstrating the robustness of the dual-stage optimization approach.

Algorithm	Llama-3.1-8B		Qwen-3-8B	
	HotpotQA	**BioASQ**	**HotpotQA**	**BioASQ**
`HEBO`	$0.500_{\pm 0.075}$	$\mathbf{0.265}_{\pm \mathbf{0.017}}$	$0.485_{\pm 0.110}$	$0.248_{\pm 0.016}$
`CDS4RAG(HEBO)`	$\mathbf{0.562}_{\pm \mathbf{0.028}}$	$0.261_{\pm 0.013}$	$\mathbf{0.594}_{\pm \mathbf{0.036}}$	$\mathbf{0.255}_{\pm \mathbf{0.005}}$
`BO`	$0.462_{\pm 0.125}$	$\mathbf{0.255}_{\pm \mathbf{0.020}}$	$0.359_{\pm 0.158}$	$0.254_{\pm 0.006}$
`CDS4RAG(BO)`	$\mathbf{0.514}_{\pm \mathbf{0.089}}$	$0.240_{\pm 0.014}$	$\mathbf{0.602}_{\pm \mathbf{0.032}}$	$\mathbf{0.260}_{\pm \mathbf{0.008}}$
`TPE`	$0.482_{\pm 0.041}$	$0.261_{\pm 0.016}$	$\mathbf{0.553}_{\pm \mathbf{0.117}}$	$0.245_{\pm 0.020}$
`CDS4RAG(TPE)`	$\mathbf{0.544}_{\pm \mathbf{0.025}}$	$\mathbf{0.273}_{\pm \mathbf{0.011}}$	$0.537_{\pm 0.042}$	$\mathbf{0.254}_{\pm \mathbf{0.010}}$

Table 2: RQ1 comparison between `CDS4RAG` and corresponding single-stage algorithms. Each cell represents mean$_{\pm \text{std}}$ of F1-score.The bold indicates the better algorithm of each case.

Algorithm	Llama-3.1-8B		Qwen-3-8B	
	HotpotQA	**BioASQ**	**HotpotQA**	**BioASQ**
`CDS4RAG(HEBO)`	$\mathbf{0.562}_{\pm \mathbf{0.028}}$	$\mathbf{0.261}_{\pm \mathbf{0.013}}$	$\mathbf{0.594}_{\pm \mathbf{0.036}}$	$\mathbf{0.255}_{\pm \mathbf{0.005}}$
`RayTune`	$0.482_{\pm 0.041}$	$0.261_{\pm 0.016}$	$0.553_{\pm 0.117}$	$0.245_{\pm 0.020}$
`Random`	$0.502_{\pm 0.049}$	$0.254_{\pm 0.018}$	$0.476_{\pm 0.115}$	$0.250_{\pm 0.012}$
`Greedy`	$0.498_{\pm 0.110}$	$0.237_{\pm 0.016}$	$0.448_{\pm 0.145}$	$0.246_{\pm 0.019}$
`AutoRAG-HP`	$0.426_{\pm 0.069}$	$0.244_{\pm 0.013}$	$0.410_{\pm 0.164}$	$0.242_{\pm 0.017}$

Table 3: RQ2 comparison with other state-of-the-art algorithm for RAG hyperparameter optimization. Each cell represents mean$_{\pm \text{std}}$ of F1-score.The bold indicates the better algorithm of each case.

D Discussion

D.1 Precision vs. Richness in Retrieval Stage

Indeed, when optimizing in the retrieval stage, there can be a trade-off between precision and richness, but their relationship is not strictly conflicting: there exist "sweet point" hyperparameter configurations that have both better precision and richness than the others, beyond which point biases to either might cause a significant drop in the generation quality. The dual-sequential formalization in `CDS4RAG` is designed to better find those "sweet points", and the cyclic iteration/refinement is purposely designed to implicitly mitigate the unwanted compromise in the trade-off: if in one cycle we bias too much towards precision (over-compromising richness), the generation quality under those related hyperparameter configurations would have worse ranks, hence making them less likely to influence the subsequent optimization.

D.2 Ground Truth and Metrics

`CDS4RAG` needs ground truth (for precision and F1-score), but we would like to kindly stress that this has been widely used in RAG hyperparameter optimization, given the vast types of benchmarks available. However, `CDS4RAG` is actually compatible with any metrics, which can be flexibly replaced. Practically, the suggested self-supervised or LLM-as-a-judge can be directly used in `CDS4RAG` when the ground truth is unavailable.

Indeed, the token-level F1-score does not cover every aspect of RAG quality. We use it here because it is a widely-applied standard of the QA-style benchmarks for RAG, enabling fair comparison with prior work, and provides a consistent scalar final optimization target. Again, other metrics such as faithfulness and answer relevance can be seamlessly applied (note that the context precision has been used as the objective for hyperparameter optimization at the retrieval phase in `CDS4RAG`).

E Reproducibility

All experiments use fixed random seeds for reproducibility. To support open science, the data, scripts, and full experimental results can be found in our anonymous repository at: https://github.com/ideas-labo/cds4rag/.